Contents

acp-cockpit user manual

acp-cockpit is a local, browser-based client for AI coding agents that speak the Agent Client Protocol (ACP). It runs a small Python server on your machine, spawns the agent's ACP adapter as a plain subprocess, and renders the structured conversation — messages, thinking, tool calls with diffs, permission requests, the agent's own questions — in your browser. Nothing is screen-scraped; everything you see comes from typed protocol data, and every byte that crosses the wire is recorded. On screen the client calls itself **ClaudIU** by default; that name is one line of configuration (see *The name it shows*).

Installing and starting

```
npm install -g @agentclientprotocol/claude-agent-acp   # the Claude Code adapter
pip install acp-cockpit                                # Python >= 3.11
acp-cockpit                                            # or: python -m acp_cockpit
```

From a checkout, `pip install -e .` gives the same program. The wheel ships everything the server needs — the page, the pinned protocol schema, the agent profiles — so an installed copy and a checkout behave the same.

Options: `--port N` (default 0 = OS-assigned), `--profiles DIR` (default: the profiles shipped in the package, then your own `~/.acp-cockpit/agents/` on top), `--records DIR` (default `~/.acp-cockpit/records`), `--records-keep-days N` (delete records older than N days at startup; default 0 = keep them all), `--no-drift-online` (skip the online version check), `--archiver FILE` (path to claude-session-publisher's `transcript_archiver.py`; also `ACP_COCKPIT_ARCHIVER`), `--ui-name-file FILE` (a TOML file holding the on-screen name), `--token-file FILE` (keep the auth token across launches).

The server prints a URL with a token — open it; the token becomes a cookie and the address bar cleans itself. By default each launch gets a fresh token and old URLs die with the server; with `--port` and `--token-file` together the URL is stable and can be bookmarked (the file is created owner-only where the OS supports it — treat it like a password).

Tabs

Each session is a tab (Alt+1…9 switch, Alt+N opens the launcher, × closes and ends the session). The tab shows the agent's own title for the session once it sets one, and a dot: green ready, pulsing blue working, red failed or disconnected. Its tooltip names the profile, the directory, the state and — once the adapter has introduced itself — the agent and version behind it. The tabs scroll; the account chips, the update chip, help and settings stay pinned to the right whatever happens. As tabs multiply their labels shrink, the way a browser's do, and past that the strip scrolls and the oldest go out of sight.

The status strip, selectors and composer always belong to the active tab; each tab keeps its own reading position when you switch. Approval dialogs and questions from any tab pop up wherever you are, labelled with the session they belong to.

The launcher

Pick an **agent** (from the profiles; an agent whose adapter is not installed shows "adapter missing" with the install hint) and a **project directory** — the session's working directory and, importantly, its *file-access boundary*: the agent can only read and write inside it through this client. Type the path or click **Browse…**: a folder picker lists the subdirectories of the current path (drive roots are one click away, **↑ Up** goes to the parent) and **Use this folder** fills the field.

Known caveats of the selected agent are listed on the right, followed by the **runtime** the adapter will be pointed at: for Claude, `CLAUDE_CODE_EXECUTABLE` resolves to your installed `claude` when there is one on `PATH`. The adapter bundles its own, older Claude CLI, which the API may refuse for newer models ("version 2.1.251 or newer is required" was the 2026-09-01 symptom); with no `claude` installed the bundled copy is used.

Thinking is chosen here — summarized, omitted, or off — because ACP fixes it when the session is created. *Off* requests no thinking tokens at all for that session.

Find resumable sessions asks the agent which of its own sessions exist for that directory (a throwaway adapter is started and closed for the query; it leaves no record) and lists them newest first, each with when it was last touched ("2 h ago", "yesterday 14:02"; the session id is in the tooltip; one the agent never dated says so and sorts last). **Resume** replays the conversation so far — prompts, answers, tool calls — before the composer enables (for the rare agent that can only attach without history, the pane starts empty). Only one attachment per agent session is allowed: resuming one that is already open somewhere is refused, naming the tab that holds it — two adapters attached to one agent session would write the same transcript file.

The launcher remembers the agent, the directory and the thinking choice you last started with — in this browser, and on the server (`/api/settings`) for a new browser or a cleared profile. Preferences, not state.

The conversation

- **Agent text** streams as it is produced; *thinking* appears dimmed and italic; your prompts are boxed. Thinking is the model's own summary of its reasoning — the API offers summaries or nothing for recent models — and a turn with only a `· thinking ·` marker had no summary to send.
- **Tool calls** are one row per call, updated in place as the agent reports progress (pending → in progress → completed/failed, colour-coded edge). Edits arrive as real diffs (+/− lines); text output and file locations open under "details".
- **Hierarchy, like the terminal**: text the agent produces *before* a tool call is a step and renders attenuated; the final answer is bright. Tool rows are dim with a status dot (blue pending, green done, red failed).
- **Markdown-lite**: `**bold**` is highlighted in the accent colour, `` `code` `` and fenced blocks are monospaced, headings stand out. The text is rendered as DOM nodes, never as HTML.

Settled lines are rendered once and never touched again, so you can select and copy an answer while it is still arriving.
- **Tool output is collapsed by default** (as in the terminal); the "tool output" checkbox in the panel under the prompt opens every result and is remembered by the browser; each row's "details" toggle always works.
- **Plan** entries (the agent's todo list) fill the panel above the composer.
- **Turn ends** are marked with the protocol's stop reason.
- **Things the agent said that this client cannot draw** — an image block, a resource, a notification it does not act on — are `unrecognized` rows (harness lane) with the raw frame one click away. Update kinds the adapter is known to send outside the ACP schema (a subagent being spawned, a background task's progress) are rows of their own, named after the kind, in the subagents or events lane.
- The `{}` button on special rows reveals the raw protocol frame behind them — the escape hatch into the session record.

The conversation follows new rows only while you are at the bottom. Scroll up to read and it stays where you put it; a **↓ jump to latest** button appears and puts you back in the stream.

Prompting

Enter sends; Shift+Enter inserts a newline. The Send button is enabled only while the session is ready (not during the agent's turn, not before startup completes). **Stop**, or **Esc**, cancels the current turn. Typing `/` first opens the command palette listing the agent's own slash commands; picking one only fills the composer — nothing is sent until you press Send, and the agent's response to a command is always rendered.

Prompt suggestions. After a turn the agent can offer a guess at what you might ask next. It appears as a dashed strip over the composer: click it to put the text in the box — **it is never sent for you** — or dismiss it with the ×. It disappears when a turn starts, and when you send anything. Most adapters never send one: `claude-agent-acp` (0.73 through 0.75.1) neither asks the SDK for suggestions nor forwards the message, so with the stock adapter the strip simply never appears. A patched adapter that forwards them puts the prediction on `_meta._claude/promptSuggestion` of an otherwise empty message chunk, and this client renders that.

The panel under the prompt

One panel holds every session control:

- the **selectors** — mode, model and whatever other configuration options the agent advertises (the Claude adapter offers mode, model, effort and agent). Each thing appears once — a config option replaces the legacy select for the same thing — and long descriptions are tooltips. The choices are offered in the order the agent's profile asks for rather than the order the agent happens to send: the Claude profile lists the models by decreasing capability (default, Fable, Opus, Sonnet, Haiku). Nothing is renamed or left out — see `config_option_order` in `PROFILE-SCHEMA.md`.
- the **five lane switches** — thinking, tools, subagents, events, harness. Switching one off removes those rows from the page entirely (they are still recorded, and switching it back on brings them back); the agent's answers and your own prompts are never hidden. The choice is remembered per browser. Thinking is the one lane that is more than a view filter: what the agent is *asked* to produce is chosen in the launcher.
- the **tool output** checkbox, and **Archive**, **Stop**, **Send**.

For the first seconds of a session the panel shows a single line — *starting the agent…* — instead of its controls. A session announces itself in pieces and putting each on screen as it landed made the panel shuffle; it waits until there is something settled to show.

Approvals

When the agent wants to do something that needs permission, a dialog shows the tool call (with diff when provided) and the agent's own options — allow once, allow always, reject — as buttons (digits 1–9 work too). There is no Escape-to-dismiss: an explicit choice is required. The options and their number come from the agent: Claude offers a standing "Always Allow" for some tools only, so a third button appears only when the agent offers a third option. Standing grants are listed after the one-shot choices and drawn dashed/amber; they are never disabled.

A tool call that names a path **outside** the project directory shows a prominent warning listing those paths: shell commands run by the agent itself are not confined by this client, so your answer is the only control. URLs and route-like tokens (`/api/...`) are not paths and raise no warning.

Unanswered requests are auto-rejected after a timeout (default one hour) and the events lane says *auto-rejected*. When the agent withdraws its own request (it cancelled the tool call), the dialog closes by itself and the events lane says *withdrawn by the agent*.

Questions from the agent

The agent can ask you a multiple-choice question instead of guessing (its AskUserQuestion tool). It arrives as a form: single-answer questions are radio buttons, multi-answer ones checkboxes, and every question has an **Other** box for an answer in your own words, which wins over the options. **Skip** answers nothing — the agent is told you skipped and carries on; the same happens by itself if the question goes unanswered past the timeout. Your answer is written into the conversation so the transcript shows what you chose. An option's *preview* (the mockup the terminal shows on focus) is not displayed here; its description is. A question the agent withdraws closes by itself and says so.

The status strip and its chips

The strip shows the session state, the current permission mode, a **context gauge** (tokens used / window size, percentage, and cost when the agent reports it) and the current model — as the API's own id (`claude-opus-5`) once the agent has reported one, with the selector's label in the tooltip.

While a turn runs, a pulsing "agent working… 42s · last activity 3s ago (tool_call: Edit hello.py)" row sits at the end of the conversation — the elapsed time tells you the turn is alive, the last-activity part tells you whether the agent is still producing events (it turns amber after a minute of silence).

Three chips can appear in the strip; the first two should not be ignored:

- **drift** — the agent sent protocol data newer than this client's pinned ACP schema (hover for details). The client keeps working and keeps recording; the flag means an update to the client is due.
- **anomalies** — something out of order happened (malformed frame, adapter crash, rejected command, an agent asking for a capability this client never advertised, an agent that wants an authentication this client does not implement…); the conversation shows the details inline with raw-frame access.
- **log (n)** — the adapter's own log (its stderr: one `[session/query] …` line per session, sometimes echoed command output). These are diagnostics, not errors; real problems come as anomalies. Click to open the drawer under the strip. Never shown inline, never dropped — it is in the record.

Clicking **anomalies** or **drift** opens a drawer with every entry, kept until you press **dismiss all**. Looking at them never throws them away.

The top right: account limits, versions, help, settings

The agent reports your **account's** rate-limit windows as it works — the five-hour window, the seven-day one, per-model windows (`7d Fable`, `7d Opus`) as soon as the agent reports them, `7d+EC` for the seven-day window with extra credits included. Each window the agent has mentioned appears as a chip with how much of it is used (`utilization` is a fraction, shown ×100); hovering gives the status, when it resets, and the agent's own payload. **EC** is extra credits: green when the account may spend them, red when it may not — the state the agent reported is in the tooltip. Windows nobody reported are not shown: this view never invents a number it was not told. The chips appear only once the agent sends a rate-limit update, which it does as your account approaches a window. **How many credits are left, in money, is not shown because the agent never sends it**: the rate-limit payload carries the *state* of extra credits and never a balance or a currency.

When the adapter announces the account it is running as (the Claude adapter does, from 0.75.1), a chip shows the plan's label — *Claude Pro* — with the e-mail, organisation and plan in its tooltip, so a screenshot of the page carries no address unless you hover.

update appears only when the pinned protocol schema or the installed adapter is behind the latest published release; it opens Help, whose **Versions** entry names the pinned schema, the installed and latest adapter versions, and whether the online check was on. The check asks `api.github.com` and `registry.npmjs.org` once per page load and sends nothing but the request; `--no-drift-online` turns it off, and Help then says so.

? opens a short guide to everything on screen. holds the settings that belong to the browser rather than to a session: the **theme** (follow the system, dark, or light) and whether tool output starts open. Session options — which agent, which directory, what thinking to ask for — are chosen per session in the launcher (the **+** tab).

Keyboard

Every shortcut there is:

Key	What it does
`Esc`	Stops the agent mid-turn. With the command list open it closes that first; inside a dialog it belongs to the dialog.
`Enter`	Sends the prompt.
`Shift+Enter`	Starts a new line instead of sending.
`/`	In an empty composer, opens the list of the agent's own slash commands.
`Alt+N`	Opens the launcher for a new session.
`Alt+1` … `Alt+9`	Switches to that tab.
`1` … `9`	Answers the open approval dialog (its buttons are numbered).
any other key	Types. Press one with the focus anywhere on the page and the character goes to the composer, the way a terminal always types at the prompt. A focused button keeps its own keys: Space presses it.

An approval dialog is the one place `Esc` does nothing: it will not be dismissed unanswered.

Archiving a conversation

Archive opens a panel beside the conversation — it never covers it, and it closes. Choose where to save and which formats:

- With **claude-session-publisher** configured (`--archiver <path>` or `ACP_COCKPIT_ARCHIVER`), you get its full document: HTML, Markdown, text, LaTeX or PDF, with the fidelity report that proves nothing was dropped. The destination reaches it as `--archive-dir`.
- Without it, the server writes a **plain Markdown transcript** from the session's own record — prompts, answers, thinking and tool-call titles — and says so, in the panel and in the file itself. A missing optional tool is a reason to write less, not a reason to refuse to save. Nothing is copied into this project.

The destination is remembered per browser. What was written is listed in the panel and noted once in the conversation's harness lane, so the record of what you did sits with the rest of the session's history.

Keeping your sessions

Sessions live in the server, not the page: reloading the browser (or opening a second window) reattaches to everything still running, with the conversation replayed once, and lands you back in the tab you were in.

If the browser loses its connection — the server restarted, the machine slept — the tab reconnects on its own and asks only for what it missed; the session itself never stopped. While it is trying, the status strip says *reconnecting*. Two things it will not retry: a refused login and a session the server no longer has, both of which say so and ask you to reload. A session that failed or was closed stays in the list for ten minutes, so you can read why, and then leaves it.

Stopping the server ends every session and every adapter: the adapter and the agent CLI it started are one process tree, closed together — by Ctrl+C, by an ordinary exit, and on Windows even by a hard kill of the server process.

Records

Every session appends to `<records-dir>/<session>.jsonl`: each protocol frame verbatim (before any interpretation), the adapter's stderr, and every client action — prompts, permission answers, file-access decisions with the policy that made them, what was launched, which runtime it was pointed at, and whether the adapter's process tree is guarded against a hard kill of the server (`tree_guard`: a Windows job object, Linux `PR_SET_PDEATHSIG`; nothing equivalent exists on macOS, where the signal handlers do the work). This is the audit trail and the ground truth; the UI's `raw_ref` numbers index into it. Records are kept for ever unless `--records-keep-days` says otherwise, and the server says at startup how many it holds and how large they are. The probe that lists resumable sessions leaves no record.

The name it shows

The client calls itself **ClaudIU** by default. That is configuration, not code: set `ACP_COCKPIT_UINAME` in the environment, or put a `uiname.toml` with that key in `~/.acp-cockpit/` (or point `--ui-name-file` at one), and the browser tab, the launcher heading and the help dialog follow. The environment wins over the file. A name longer than 15 characters is cut with an ellipsis so it still fits a tab, and a blank or broken setting falls back to the default rather than leaving the interface nameless.

The project, the package and this manual are `acp-cockpit`; only the name on screen is yours.

Adding your own agent profile

Every `*.toml` in the profiles directories is an agent this client can start (`acp_cockpit/agents/PROFILE-SCHEMA.md` is the contract). The shipped profiles live inside the package; **yours go in `~/.acp-cockpit/agents/`**, loaded on top of the shipped ones (a file with the same `id` replaces the shipped profile). That is where a profile pointing at a local adapter build, an experiment, or a private agent belongs — a machine-specific path never reaches the repository, and an installed copy never has to touch site-packages. In a checkout, `acp_cockpit/agents/local-*.toml` does the same and is never tracked.

Security notes

The server binds 127.0.0.1 only; every request needs the launch token — the page and its files included; a Host that is not loopback, or an Origin that is not this server's own address and port, is refused (cookies are not scoped by port, so a page served from another local port would otherwise arrive authenticated). Agent file access is confined to the project directory (symlinks resolved); a ranged read (`line`, `limit`) returns only the range asked for; a file that cannot be read as text is answered with an error, never left hanging. An agent that asks for a capability this client never advertised (a terminal) is refused and the refusal is shown. Adapter subprocesses run with a scrubbed environment (plus the profile's declared runtime resolution and settings, written as the first record of the session) and die with their session. The record files never contain your token — but they do contain your conversation, so treat the records directory accordingly.

This client does not implement ACP `authenticate`. An agent that requires it is reported as an anomaly at start-up (naming the methods it offered) rather than failing with a bare error.